Build powerful solutions with Ruby inheritance and NOSQL databases

By

Reid Morrison

Software Architect

Clarity Services, Inc.

Ruby and NOSQL databases

Lets build a batch processing system in Ruby

- Problem Statement
- Single Collection Inheritance
- Batch Processing Requirements
- Data Store Requirements
- rocketjob
- What's next?
- Questions?

Who a

- About me
- About rocketjob project
  - Enterprise Batch Processing System
  - Re-design after re-design
  - 1 year old
- Call it RocketCat ?

Problem Statement

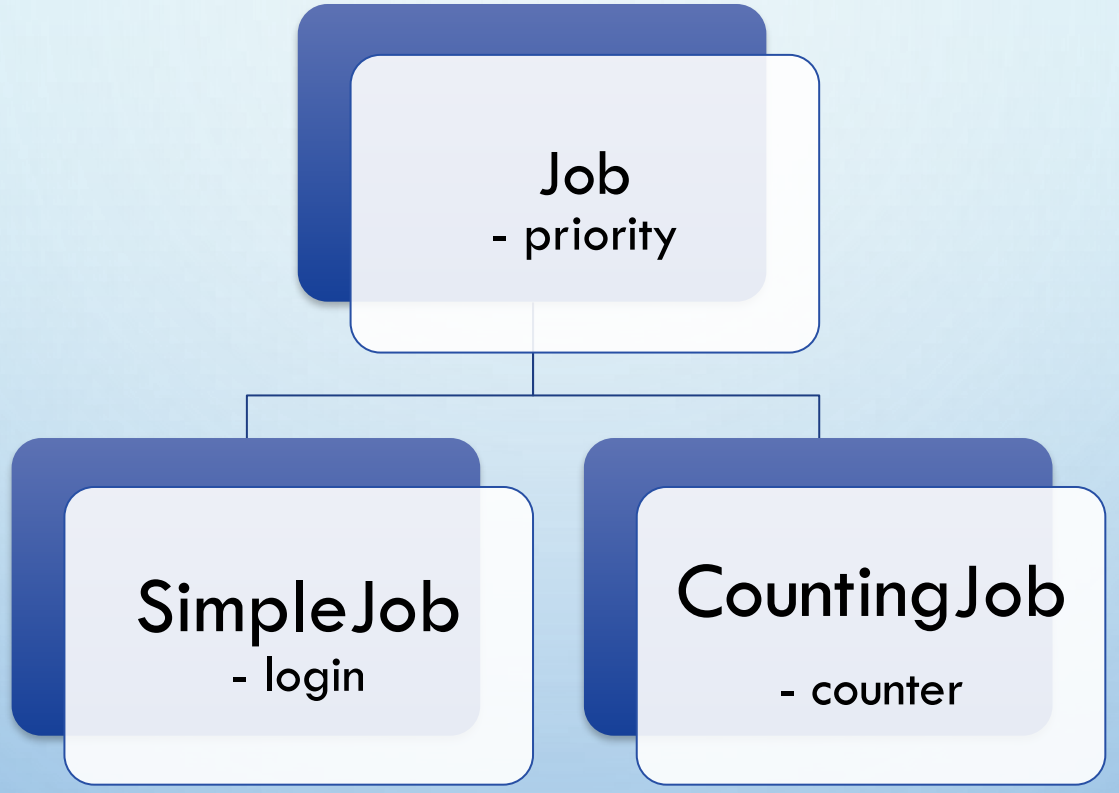
At Clarity, we often run:

- 100 million business critical transactions in batch per day.
- 0.1 to 12.0 seconds per transaction.
  - Depends on the basket of products requested.
- Business creates new transaction types and jobs frequently
  - Around 10 different kinds in play at any time.
- Business needs to be able to:
  - Change the priority of each job while it is running.
  - Pause, resume, or abort active jobs.
  - View and control every job in the system.
    - Self-service support!
- Support for very large input and output batch files.
- As a credit bureau everything needs to be PCI compliant.
  - Encrypt everything at rest and in flight.

Problem State

- Resque & Sidekiq-pro in production.
- Needed far more than just background job processing.
  - Needed an entire management and control system wrapped around a batch environment.

Lets build it



- Inheritance class hierarchy
- Store in single collection `*jobs*`

What is Single Collection Inheritance (SCI)

- Store all objects in an inheritance class hierarchy in a single collection.
- Document oriented NOSQL database
- No Database Migrations
- Each document contains only the attributes needed for that model

Class	Attributes
Job	priority
SimpleJob	priority, login
CountingJob	priority, counter

Example

```
class Job
  include MongoMapper::Document
  key :priority, Integer, default: 50
end

class SimpleJob < Job
  key :login,    String
end
```

- No database migrations!

Queue jobs for processing

```
Job.create!(
  priority: 50
)

SimpleJob.create!(
  priority: 20,
  login:    'jbloggs'
)
```

Lookups

Job.count

 # => 40

SimpleJob.count

 # => 3

- Job & SimpleJob are stored in the `*jobs*` collection
- Job.count
  - All jobs in the system
- SimpleJob.count
  - Only count SimpleJobs

Lookups

Job.first

- Fetch the first Job

SimpleJob.first

- Fetch the first SimpleJob

SimpleJob.*where*(login: **'admin'**).first

- Fetch the first SimpleJob with login `*admin*`
  - Parent Job does not have or need `*login*` attribute

SimpleJob.*where*(login: **'admin'**).count

- Returns the count of SimpleJob with login `*admin*`

Adding behavior

```
class Job
  include MongoMapper::Document
  key :priority, Integer, default: 50

  def perform
    fail(NotImplementedError, 'must implement #perform')
  end
end

class SimpleJob < Job
  key :login,   String

  def perform
    # Run this simple job
  end
end
```

- SimpleJob implements its own #perform method

Queue job with behavior for processing

```
SimpleJob.create!(
  priority: 20,
  login:    'jbloggs'
)
```

- Data is stored in NOSQL
- Behavior is held in specialized class SimpleJob

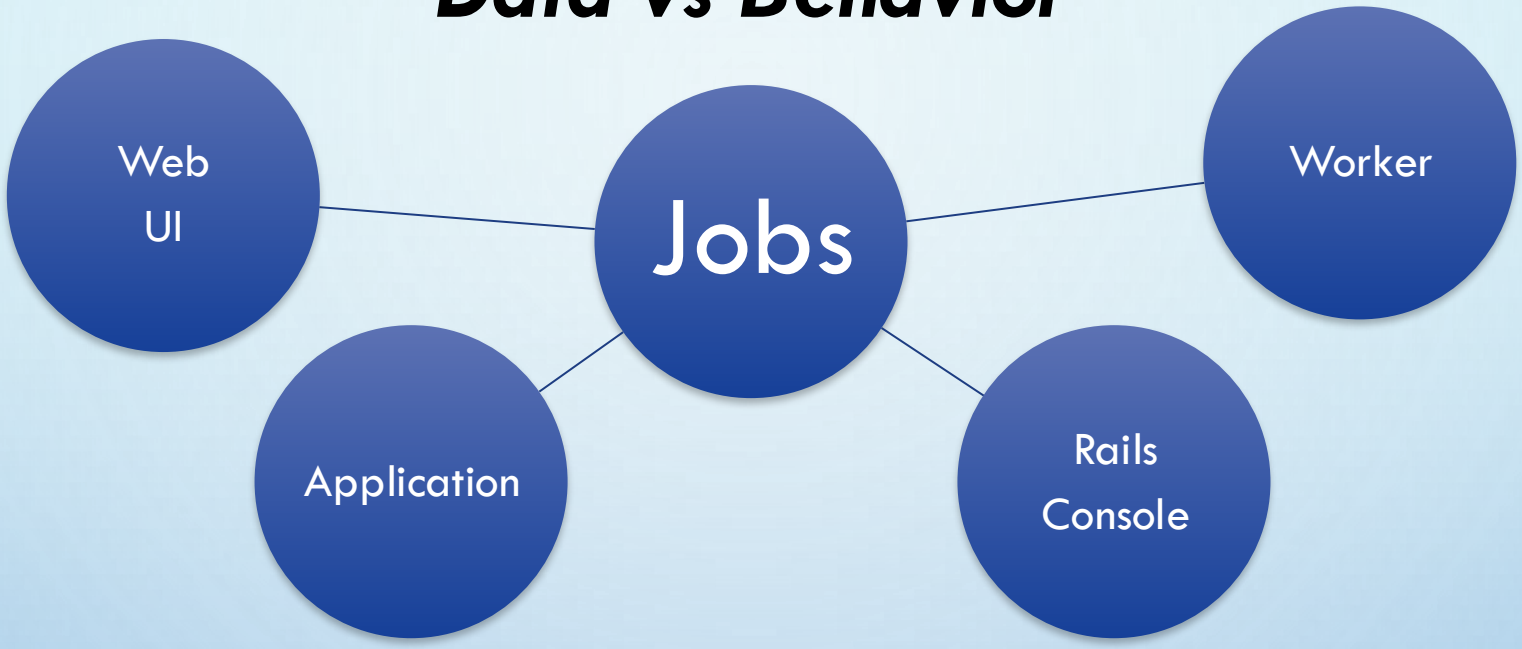
Workers process any job in the collection

```
# Sample Worker:

while job = Job.first
  job.perform
  job.destroy
end
```

- Processes any child class of Job
- Any behavior can be implemented in child class

Data vs Behavior



- Job data is in only one place
  - State: :running, :complete, etc.
  - Arguments
  - Result
  - Immediately available
- Behavior is everywhere

Specialize parent behavior

```
class Job
  # Returns [Hash] user readable status for this job
  def status
    { priority: priority }
  end
end

class SimpleJob < Job
  key :login, String

  # Add login to this Jobs status
  def status
    h = super
    h[:login] = login
    h
  end
end
```

- SimpleJob adds to Jobs behavior

Schedule job for processing

```
job = SimpleJob.create!(
  priority: 20,
  login:    'jbloggs'
)
```

- Now check on the jobs status

```
job.reload.status
# => { priority: 20, login: 'jbloggs' }
```

- Specialization using SCI adds behavior
- Use `*reload*` to get a fresh copy of the job

Child classes can specialize the behavior

```
# Base class implementation for processing jobs
class Job
  # Runs this job
  def work
    before_perform
    perform
    after_perform
    destroy
  end

  def perform
    fail(NotImplementedError, 'must implement #perform')
  end

  def before_perform
  end

  def after_perform
  end
end
```

Specialize how work is performed

```
class BatchJob < Job
  key :lines, Array

  # Upload work for this job
  def upload(filename)
    File.open(filename, 'rt') do |file|
      file.each_line do |line|
        lines << line
      end
    end
  end

  def work
    before_perform
    # perform is called many times
    lines.each do |line|
      perform(line)
    end
    after_perform
    destroy
  end

  def perform(line)
    fail(NotImplementedError, 'must implement #perform')
  end
end
```

New #upload

Specialized #work

Specialized BatchJob to process a large file

```
class AmazingBatchJob < BatchJob
  def perform(line)
    # process the line
  end
end
```

- The perform method is called once for every line
- Only one job is created
- Job holds the data to be executed

Queue for processing

```
job = AmazingBatchJob.new(
  priority: 10
)
job.upload('file.csv')
job.save!
```

- Queue the job for processing
- Upload the file before starting processing

Specialized behavior

```
# Process 2 lines at a time
class AnotherBatchJob < BatchJob
  # Replace the standard upload method
  def upload(filename)
    File.open(filename, 'rt') do |file|
      # Every 2 lines make up a single record
      file.each_slice(2) do |lines|
        save(lines)
      end
    end
  end

  def perform(lines)
    # process 2 lines at a time
  end
end
```

Specialized behavior …

```
# Each perform now gets 2 lines at a time


job = AnotherBatchJob.new
job.upload('file.csv')
job.save!
```

Worker Concurrency

```
# Can only run one worker at a time
while job = Job.first
  job.work
  job.destroy
end
```

- Multiple workers would process the same job
- Need to:
  - Distinguish between queued and running workers
  - Assign the job to the current worker

Worker Concurrency

- Set the initial state of any job to :queued
- Allow the worker_name to be set for any job

```
class Job
  include MongoMapper::Document
  key :priority,    Integer, default: 50

  key :state,       Symbol, default: :queued
  key :worker_name, String

  def perform
    fail(NotImplementedError, 'must implement #perform')
  end
end
```

Update in place

```
# Update a job in place and assign it to the current worker
while job = Job.find_and_modify(
    {state: :queued},                                  # find
    {state: :running, worker_name: host_name} # modify
  )
    job.work
end
```

- Find the first :queued job
- Set *state* to *:running* and *worker_name* to *host_name*
- Change is immediately visible
- Processes any job
- Supports thousands of workers
  - same *find_and_modify* call
  - concurrently

Finished!

- Background Job processing system is now complete
- Single collection inheritance
  - Stored in a single collection
  - Specialized behavior
  - Extensible
  - Fast!

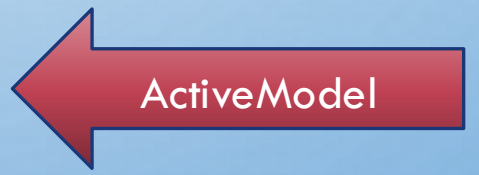
Validations

```
class Job
  include MongoMapper::Document
  key :priority, Integer

  validates :priority, inclusion: 1..100
end

class SimpleJob < Job
  key :login, String

  validates_presence_of :login
end
```



Batch Processing System Requirements

- Visibility
  - See every job in Web UI
  - API access to every jobs status
- High Performance
  - Low overhead
- High Availability
  - Automatic failover
  - No single point of failure (SPOF)
- High Volume
  - Millions of records
  - Large amounts of data
- Self-Service Support
- PCI Compliance
  - Security
  - Encryption

Data Store Options

- Relational Databases
  - MySQL, Postgres, etc.
  - Severe locking under high load
  - Poor update in place support
  - Delayed::Job

Data Store Options …

- Messaging & Queueing
  - redis, IBM MQ, ActiveMQ, RabbitMQ, 0MQ, Beanstalkd, HornetQ
  - Data all over the place
    - Job request in the queue
    - Status?
    - Output data?
    - Large input data?
  - resque, sidekiq

Data Store Options …

- Document Oriented NOSQL Database
  - Store everything in one document
  - Update in place
  - MongoDB, Cassandra, CouchDB, HBase, Accumulo, Hypertable
    - http://kkovacs.eu/cassandra-vs-mongodb-vs-couchdb-vs-redis
  - rocketjob

Data Store Requirements

- High Performance
  - In-memory
  - Disk overflow for large data
  - Update in place
    - find_and_modify
- High Availablity
  - Automated failover
  - Persistent
- Security
  - Authentication
  - Authorization
- Supported

We chose:

Processes vs Threads

Process	Thread
Memcache to share across processes	Shared Memory
Heavyweight	Lightweight
Forking issues	Concurrency issues
Limited	Scalable
Middleman process	No middleman
Identify bad jobs	One process for all threads
Ruby MRI	Recommend JRuby
	Management Thread - heartbeat - config updates - remote start/stop

rocketjob - Batch Processing System

- Priority based job processing
- Mission Control
  - Web UI
  - Visibility of all jobs
  - Retry failed jobs
  - Manage Workers
- Callbacks for every state transition
  - For example, send an email at every step, or on completion
- Directory Monitor
  - Enqueue jobs when a file arrives in a directory

rocketjob - Batch Processing System …

- Future dated jobs
  - run_at
  - Decentralized scheduled job system
- Singleton Jobs
- Deployment
  - Jobs will complete on old code base
  - New jobs use new codebase
- Does not require Rails
- Over a year of full time development effort

rocketjob pro - Batch Processing System …

- Batch Jobs
  - Upload large files directly into the job
    - Supports multiple input and output files
  - Encryption and compression of uploaded data
  - Stores both the uploaded file and output data
  - Change priority on the fly
  - Collect job output
  - Large file support
    - Billions of records
    - Streaming of very large files
    - File compression and encryption

rocketjob pro - Batch Processing System …

- Mission Control
  - Pause, Fail, Abort batch jobs
  - Change job priority
  - Change job worker limit
    - Prevent overwhelming databases or other systems
- Fully utilize available hardware
  - Breaking large jobs into smaller "slices"
  - All available servers can work on "slices"
- Deployment
  - In progress Batch Jobs "paused" during deployment
  - Resumed when workers restart on new codebase

What's next?

- Easier Job scheduling
- ActiveJob
- Capistrano
- Other job processing patterns
  - Map-reduce
  - Composite jobs
  - Output from one Batch job is input into the next job
- etc...

Questions?

- Feedback?
- We want your help!
  - github.com/rocketjob/rocketjob
- Star project on github

http://rocketjob.io